Dataset Distillation

Vasilii Silvestrov[1] Daria Linichenko[1] Kseniia Tsypliakova[1] Ilya Larin[1]
Aleksandr Klimchuk[1]

June 3, 2026

HSE University, Moscow, Russia
{vasilvestrov, dalinichenko, kdtsypliakova, ialarin, aaklimchuk}@edu.hse.ru
Our implementation is available at https://github.com/runtime57/dataset-distillation

Contents

Abstract

Dataset distillation aims to replace a large training set with a small number of synthetic examples on which a model can be trained almost as effectively as on the original data. For images, a synthetic example can be optimized directly as a continuous tensor of pixels, but in text the input consists of discrete token IDs, which is not directly compatible with standard gradient-based optimization. The standard workaround is a soft-token relaxation: at each position we keep a distribution over the vocabulary and feed a weighted sum of embeddings into the model. This approach is differentiable, but it scales poorly, requiring $O(MT|\mathcal{V}|)$ parameters for M synthetic texts of length T. In this project, we propose and experimentally compare four parameterizations of synthetic text: a full soft-token baseline without dimensionality reduction, a naive top-k sparse variant, an anchor-based factorization of vocabulary distributions, and a concept-based parameterization in a compact feature space. The goal is to understand whether text dataset distillation quality can be preserved while substantially reducing the number of parameters and narrowing the gap between soft representations and discrete text.

1 Project Idea and Overview

1.1 Context

Dataset distillation compresses a training dataset not by selecting a subset of real examples, but by synthesizing a small set of training instances. For images, this is natural: a synthetic image is a continuous pixel tensor through which gradients can be computed directly. In text, the situation is fundamentally harder: an example is a sequence of tokens

$$x = (w_1, \dots, w_T), \qquad w_t \in \{1, \dots, |\mathcal{V}|\},$$

and each token ID is discrete. This leads to the main question of the project:

How can we parameterize synthetic text so that it can be optimized with gradients while remaining compact and decodable into ordinary tokens?

We take soft-token relaxation as the baseline approach: instead of a single token ID at position t, we keep a distribution $p_t \in \Delta^{|\mathcal{V}|}$, and the input embedding is computed as

$$\widetilde{e}_t = p_t \mathbf{E}, \qquad \mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}.$$

If p_t is one-hot, this is the usual embedding lookup. If it is soft, it becomes a differentiable "mixture of tokens." The problem is that for M synthetic sequences of length T, we must store a tensor of logits $A \in \mathbb{R}^{M \times T \times |\mathcal{V}|}$. For example, with $M = 100$, $T = 128$, and $|\mathcal{V}| = 32000$, this already amounts to 409.6 million parameters for the synthetic dataset alone.

1.2 What We Propose to Study

The project is not limited to a single heuristic. We aim to build a unified experimental framework and compare four families of parameterizations:

1. **Full soft-token baseline.** For each position, we learn a full logit vector $a_{m,t} \in \mathbb{R}^{|\mathcal{V}|}$ and the distribution $p_{m,t} = \mathrm{softmax}(a_{m,t}/\tau)$. This is the most expressive variant and serves as the main baseline.

2. **Top-k sparse tokens.** For each position, we allow nonzero mass only on a set $\Omega_{m,t}$ of size $k \ll |\mathcal{V}|$:

$$p_{m,t,v} = 0 \quad \text{for } v \notin \Omega_{m,t}, \qquad p_{m,t,\Omega} = \mathrm{softmax}(b_{m,t}/\tau), \quad b_{m,t} \in \mathbb{R}^k.$$

   This is a simple test of the hypothesis that most of the full soft-token mass is redundant.

3. **Anchor-based parameterization.** We introduce K global anchor distributions $r_j \in \Delta^{|\mathcal{V}|}$ and learn only a mixture of anchors at each position:

$$p_{m,t} = \sum_{j=1}^{K} \alpha_{m,t,j} r_j, \qquad \alpha_{m,t} \in \Delta^K, \quad K \ll MT.$$

   This factorizes the full tensor of distributions while reusing "prototype soft tokens" across positions.

4. **Concept-based parameterization.** Instead of learning a distribution over the vocabulary, we learn sparse coordinates in a concept space:

$$z_{m,t} \in \mathbb{R}^K, \qquad \widetilde{e}_{m,t} = z_{m,t} C, \qquad C \in \mathbb{R}^{K \times d}.$$

For decoding, we can use the nearest token under $\langle \widetilde{e}_{m,t}, E_v \rangle$ or a learnable mapping $p_{m,t} = \mathrm{softmax}(z_{m,t}W)$, where $W \in \mathbb{R}^{K \times |\mathcal{V}|}$. Unlike anchors, concepts do not have to live in the token simplex and can instead inhabit a more compact feature space.

The main scientific hypothesis of the project is that *additional constraints on soft-token representations can substantially reduce parameter count and memory usage without a catastrophic drop in distillation quality.* At this stage, these parameterizations are introduced only as the main options we plan to compare; the literature background, formal problem statement, and detailed evaluation setup are presented in the following sections.

2 Literature Review

Dataset distillation, also referred to as dataset condensation in part of the literature, studies how to replace a large training dataset with a much smaller synthetic dataset that preserves its training utility. Unlike coreset selection, where the compressed set is restricted to real examples selected from the original dataset, dataset distillation typically treats the synthetic examples themselves as optimizable variables. As a result, the distilled samples may be images, node features, embeddings, soft labels, or other continuous objects that do not necessarily correspond to natural data points, but are optimized to induce useful learning dynamics in a downstream model.

The area was originally developed mostly in supervised image classification, where synthetic images can be optimized so that a model trained on them reaches performance close to training on the full dataset [13, 15, 1]. Since then, the same general idea has been extended beyond vision. In graph learning, graph condensation methods synthesize compact node features and, in some cases, graph structure, so that GNNs trained on the condensed graph approximate the performance of GNNs trained on the original graph [4]. Similar compression ideas have also been explored in offline reinforcement learning, where the goal is to distill a large offline dataset into a smaller training set that still supports effective policy learning [6]. In natural language processing, dataset distillation is more difficult because the input space is discrete: optimizing token sequences directly is not compatible with standard gradient-based methods.

For our project, the literature is best organized along two axes. The first axis is the distillation objective: what signal defines a good synthetic dataset. Existing methods differ in whether they optimize final validation performance after training on synthetic data, match gradients between real and synthetic batches, align hidden representations, or match longer training trajectories. The second axis is the parameterization of synthetic text itself: whether synthetic examples are represented as free embeddings, full soft-token distributions, sparse token mixtures, anchor-based mixtures, or concept-level vectors.

2.1 Objectives for Dataset Distillation

Existing dataset distillation methods differ not only in how synthetic data are parameterized, but also in which optimization objective is used to train them. Let $D = \{(x_i, y_i)\}_{i=1}^{N}$ denote the original dataset and let $\tilde{D} = \{(\tilde{x}_j, \tilde{y}_j)\}_{j=1}^{M}$ be a small synthetic dataset, where $M \ll N$. In general, the goal is to choose $\tilde{D}$ so that a model trained on synthetic data transfers well to the original task:

$$\tilde{D}^* = \arg\min_{\tilde{D}} \mathcal{L}_{outer}\big(\theta^*(\tilde{D}); D\big), \qquad \theta^*(\tilde{D}) = \arg\min_{\theta} \mathcal{L}_{train}(\theta; \tilde{D}).$$

In the original dataset distillation formulation of Wang et al. [13], this idea appears as a bilevel optimization problem: the model performs one or several training steps on synthetic data, and the outer loss is then measured on real data. If the learner starts from θ_0, the inner update can be written as

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}_{syn}(\theta_t; \tilde{D}),$$

while the outer objective takes the form

$$\min_{\tilde{D}} \mathcal{L}_{real}(\theta_K; D).$$

This objective directly measures the eventual usefulness of the synthetic set, but it requires differentiating through the learner's training dynamics, which becomes expensive when the number of inner steps grows. At the same time, this bilevel family naturally motivates a variant that we also want to test in our experiments: optimizing synthetic data for the quality of the learner after a single gradient step, while treating the step size η as a learnable parameter rather than as a fixed hyperparameter.

Zhao et al. [15] proposed a more local alternative by formulating dataset condensation as a gradient matching problem. Instead of optimizing final model quality after training on synthetic data, they require a real batch and a synthetic batch to induce similar gradients. For a real batch $B \subset D$ and a synthetic batch $\tilde{B} \subset \tilde{D}$, define

$$g_{real} = \nabla_\theta \mathcal{L}_{real}(\theta; B), \qquad g_{syn} = \nabla_\theta \mathcal{L}_{syn}(\theta; \tilde{B}).$$

The objective is then

$$\mathcal{L}_{GM} = d(g_{real}, g_{syn}),$$

where $d(\cdot, \cdot)$ is a distance between gradients. A common choice is cosine distance,

$$d(g_{real}, g_{syn}) = 1 - \frac{\langle g_{real}, g_{syn} \rangle}{\|g_{real}\| \|g_{syn}\|},$$

or squared L_2 distance,

$$d(g_{real}, g_{syn}) = \|g_{real} - g_{syn}\|_2^2.$$

If matching is performed layer by layer, the objective becomes

$$\mathcal{L}_{GM} = \sum_{\ell=1}^{L} d\left(\nabla_{\theta_\ell} \mathcal{L}_{real}(\theta; B), \nabla_{\theta_\ell} \mathcal{L}_{syn}(\theta; \tilde{B})\right).$$

Intuitively, synthetic data are good if they produce the same training signal as real data. This objective is especially attractive in our setting because it allows synthetic text parameters to be optimized directly without fully unrolling the learner's training process.

Another family of objectives compares intermediate representations rather than gradients. In distribution or feature matching methods, real and synthetic batches are encouraged to induce similar hidden-state distributions in the learner [14]. If $h_\ell(\theta; x)$ denotes the representation of input x at layer ℓ, and

$$\mu_\ell(B) = \frac{1}{|B|} \sum_{x \in B} h_\ell(\theta; x), \qquad \mu_\ell(\tilde{B}) = \frac{1}{|\tilde{B}|} \sum_{\tilde{x} \in \tilde{B}} h_\ell(\theta; \tilde{x}),$$

one can define a simple feature matching objective as

$$\mathcal{L}_{FM} = \sum_{\ell=1}^{L} \left\| \mu_\ell(B) - \mu_\ell(\tilde{B}) \right\|_2^2.$$

This objective uses a more local and less constrained signal than full trajectory matching, but can be cheaper than gradient matching because it avoids computing and comparing parameter gradients. Since our project explicitly compares distillation objectives, feature matching is a natural objective family to include in the experimental matrix.

Another line of work uses a longer-horizon signal. Cazenavette et al. [1] propose matching training trajectories: synthetic data are optimized so that after several training steps, a student model reaches a state close to an expert trajectory obtained from training on real data. Let

$$\tau^* = \{\theta_0^*, \theta_1^*, \dots, \theta_T^*\}$$

be a precomputed expert trajectory. The student model starts from θ_t^* and performs N updates on synthetic data:

$$\hat{\theta}_{t+i+1} = \hat{\theta}_{t+i} - \eta \nabla_\theta \mathcal{L}_{syn}(\hat{\theta}_{t+i}; \tilde{D}), \qquad \hat{\theta}_t = \theta_t^*.$$

The student parameters are then compared to a later point on the expert trajectory:

$$\mathcal{L}_{MTT} = \frac{\|\hat{\theta}_{t+N} - \theta_{t+M}^*\|_2^2}{\|\theta_t^* - \theta_{t+M}^*\|_2^2}.$$

Trajectory matching can be seen as a stronger version of gradient matching: instead of enforcing agreement at a single local step, it requires several synthetic updates to reproduce a trajectory similar to the one induced by real data. However, this objective requires storing expert trajectories and unrolling several learner updates, making it substantially more expensive.

For text data, an additional difficulty appears because inputs are discrete tokens, so direct optimization of $\tilde{x}$ is not directly compatible with standard backpropagation-based methods. Early text dataset distillation approaches address this limitation by optimizing synthetic examples at the embedding level [11]. This makes the process differentiable, but weakens interpretability and transferability across models because the synthetic samples become tied to the embedding layer of a specific learner. More recent text-level approaches attempt to overcome this limitation. For example, DiLM [8] trains a language model to generate synthetic training samples as ordinary text, while Textual Dataset Distillation via Language Model Embedding [12] uses language-model embeddings for a more model-agnostic text distillation pipeline.

A separate text-level line formulates the objective through reward matching. In CondenseLM [10], the synthetic dataset is generated in an LLM-driven way and optimized so that a reward reflects representability and coverage with respect to the original dataset. In general form, this can be written as

$$\tilde{D} = G_\phi(D), \qquad \max_\phi R(\tilde{D}, D),$$

or equivalently

$$\min_\phi -R(G_\phi(D), D).$$

If the reward decomposes into several components, for example coverage and representability, it can be written as

$$R(\tilde{D}, D) = \alpha R_{cov}(\tilde{D}, D) + (1-\alpha) R_{rep}(\tilde{D}, D).$$

For our project, this literature suggests that the objective should not be treated as fixed. Instead, it should be regarded as one of the two central experimental axes. We therefore want to compare gradient matching, one-step optimization with a learnable learning rate, feature matching, and trajectory matching as alternative ways of defining what makes a distilled text dataset useful. The second axis, discussed below, concerns how synthetic text is parameterized in the embedding or soft-token space.

2.2 Parameterizations of Synthetic Text Representations

A central challenge in text dataset distillation is the choice of representation for synthetic examples. Unlike images, where inputs lie in a continuous space and can be directly optimized with gradient-based methods, text inputs are discrete sequences of token identifiers. This discreteness prevents direct optimization and motivates continuous relaxations or alternative parameterizations that enable gradient flow.

A common approach is to operate in the embedding space, replacing discrete tokens with continuous vectors. This idea appears both in early work on text dataset distillation [11] and in parameter-efficient adaptation methods such as prompt tuning [5]. However, the specific choice of parameterization plays a crucial role: it introduces a trade-off between expressivity, parameter efficiency, and the ability to recover meaningful discrete text.

In this work, we focus on parameterizations of synthetic text. The objective side was discussed above; here the goal is to describe the search space for the synthetic examples themselves. In general, we write a synthetic text representation as

$$\tilde{x}_\psi = (\tilde{e}_1, \dots, \tilde{e}_T),$$

where ψ denotes the optimizable parameters and $\tilde{e}_t \in \mathbb{R}^d$ is the synthetic embedding at position t.

Full soft-token parameterization. The most expressive relaxation is the full soft-token representation, where each token position is modeled as a categorical distribution over the entire vocabulary. For each position t, logits $a_t \in \mathbb{R}^{|\mathcal{V}|}$ define

$$p_t = \mathrm{softmax}(a_t/\tau), \qquad \tilde{e}_t = p_t E,$$

where $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ is the embedding matrix and τ is a temperature parameter. Equivalently,

$$\tilde{e}_t = \sum_{v \in \mathcal{V}} p_{t,v} E_v.$$

This formulation is equivalent to optimizing directly in the convex hull of token embeddings. It can represent arbitrary mixtures of vocabulary tokens and therefore serves as a natural baseline for embedding-level text distillation. It is also conceptually related to soft prompt optimization, where learnable continuous vectors replace discrete tokens [5].

The main drawback is parameter inefficiency: the number of optimizable parameters scales as

$$O(MT|\mathcal{V}|),$$

where M is the number of synthetic examples and T is the sequence length. For large vocabularies, this becomes expensive. In addition, the learned distributions may remain high-entropy, which makes them difficult to decode into coherent discrete sequences. This creates a soft-to-hard gap: the synthetic examples may perform well as continuous embeddings but degrade after discretization.

Sparse top-k token distributions. A natural hypothesis is that effective soft-token representations do not require support over the full vocabulary. Instead, each position may be represented by a small set of candidate tokens. Let

$$\Omega_t \subset \mathcal{V}, \qquad |\Omega_t| = k, \qquad k \ll |\mathcal{V}|.$$

Only logits over this subset are optimized:

$$p_{t,v} = 0 \quad \text{for } v \notin \Omega_t, \qquad p_{t,\Omega_t} = \mathrm{softmax}(b_t/\tau), \qquad b_t \in \mathbb{R}^k.$$

The synthetic embedding becomes

$$\tilde{e}_t = \sum_{v \in \Omega_t} p_{t,v} E_v.$$

This reduces the number of parameters to

$$O(MTk), \qquad k \ll |\mathcal{V}|.$$

The approach is related to sparse probability mappings such as sparsemax and entmax, which produce distributions with exact zeros while preserving differentiability [9]. It is also compatible with differentiable relaxations of discrete choices, such as Gumbel-Softmax and Concrete distributions [3, 7]. In our setting, however, these methods are not objectives by themselves: they define the parameterization or relaxation through which the selected distillation objective is optimized.

Anchor-based parameterization. A more constrained alternative is to represent each token distribution as a mixture of a small number of shared anchor distributions. Let

$$r_j \in \Delta^{|\mathcal{V}|}, \qquad j = 1, \dots, K$$

be global soft-token anchors. For each position t, we learn mixture weights

$$\alpha_t = \mathrm{softmax}(u_t), \qquad \alpha_t \in \Delta^K,$$

and define

$$p_t = \sum_{j=1}^{K} \alpha_{t,j} r_j, \qquad \tilde{e}_t = p_t E.$$

This factorizes the full tensor of vocabulary distributions into shared prototype distributions and position-specific mixture weights. It reduces the number of parameters from $O(MT|\mathcal{V}|)$ to $O(K|\mathcal{V}| + MTK)$.

The trade-off is that anchor-based representations are less expressive than full soft-token distributions. If the number of anchors is too small, the synthetic examples may not capture the training signal needed for effective distillation. However, anchor mixtures can be more compact and easier to decode than dense full-vocabulary distributions.

Concept-level parameterization. The most compressed variant is to parameterize synthetic examples through a set of concept vectors rather than explicit vocabulary tokens. Let

$$C = \{c_1, \dots, c_m\}, \qquad c_j \in \mathbb{R}^d.$$

Then

$$q_t = \mathrm{softmax}(r_t/\tau), \qquad \tilde{e}_t = \sum_{j=1}^{m} q_{t,j} c_j.$$

This approach can be more expressive than fixed anchor distributions because the concept vectors are not constrained to lie exactly on existing vocabulary embeddings. At the same time, it is less directly interpretable: a learned concept vector may not correspond to a real token, so decoding requires nearest-neighbor projection or an additional mapping back to the vocabulary.

Soft-to-hard decoding. For all continuous or soft-token parameterizations, an important question is whether the optimized representation can be converted into discrete text. For a full soft-token distribution, the simplest decoding rule is

$$\hat{x}_t = \arg\max_{v \in \mathcal{V}} p_{t,v}.$$

For top-k variants, the same rule is applied over the restricted support:

$$\hat{x}_t = \arg\max_{v \in \Omega_t} p_{t,v}.$$

The performance difference between the continuous synthetic representation and its decoded hard-token version defines the soft-to-hard gap. Reducing this gap is one of the main motivations for these parameterizations: sparse or anchor-based distributions are expected to be easier to discretize than dense full-vocabulary mixtures.

Overall, this parameterization taxonomy is complementary to the objective taxonomy from the previous subsection. In the experiments, the objective can be fixed while changing the parameterization, or the same parameterization can be evaluated under different objectives. This separation allows us to study whether improvements come from a better distillation signal or from a more suitable synthetic text representation.

3 Problem statement

Before introducing the formal notation, we first describe the general problem that motivates this project. We consider the broader problem of data-efficient training for language models. In the standard setting, a causal language model is trained on a large text corpus to solve the next-token prediction task: given the previous tokens in a sequence, the model should predict the next one. The quality of such training is evaluated on real held-out data, usually by validation loss or perplexity.

There are different ways to make language model training more efficient. For example, one can select a smaller subset of real examples, build a coreset, change the training curriculum, compress the model, or use dataset distillation. In this project, we focus on dataset distillation as one possible approach to this broader problem. The goal is not to design a new language model architecture, but to replace the original training corpus with a much smaller synthetic dataset which still preserves the useful training signal of the original data.

Therefore, the main question is whether a compact synthetic dataset can be constructed so that a model trained on it performs close to a model trained on the full real corpus. The downstream task that defines this usefulness is causal language modeling. Distillation objectives such as gradient matching, feature matching, one-step optimization, and trajectory matching are used only as surrogate objectives for learning the synthetic data. The final success criterion is the quality of the language model trained on the distilled dataset, measured on real held-out data, together with the compactness and parameter efficiency of the distilled dataset.

In our work, we study this question specifically from the point of view of synthetic text parameterization. Since text consists of discrete tokens, synthetic examples cannot be optimized directly in the same way as continuous images. For this reason, we compare several continuous or soft-token parameterizations of synthetic text and analyze whether they can reduce the number of trainable parameters while preserving downstream language-modeling quality.

4 Formalization

4.1 Problem

Let $\mathcal{D} = \{x_i\}_{i=1}^{N}$ be the training corpus, where each sequence

$$x_i = (w_{i,1}, \ldots, w_{i,T_i}), \qquad w_{i,t} \in \{1, \ldots, |\mathcal{V}|\}.$$

Let f_ϕ be a causal language model with parameters ϕ. For a batch B of real or synthetic sequences, we write the next-token training loss as

$$\mathcal{L}_{\mathrm{LM}}(\phi; B) = -\frac{1}{|B|} \sum_{x \in B} \sum_{t=1}^{T_x} \log p_\phi(w_t \mid w_{<t}).$$

We seek a small distilled dataset

$$\widetilde{\mathcal{D}}_\psi = \{\widetilde{x}_m\}_{m=1}^{M}, \qquad M \ll N,$$

where each $\widetilde{x}_m$ is represented through continuous parameters ψ rather than fixed hard tokens. If a learner is initialized at ϕ_0 and trained on $\widetilde{\mathcal{D}}_\psi$ for K steps, its parameters evolve as

$$\phi_{t+1} = \phi_t - \eta_t \nabla_\phi \mathcal{L}_{\mathrm{LM}}(\phi_t; \widetilde{\mathcal{D}}_\psi), \qquad t = 0, \ldots, K-1.$$

The ideal goal is to choose $\widetilde{\mathcal{D}}_\psi$ so that a model trained on it achieves low loss on held-out real data:

$$\psi^* = \arg\min_\psi \mathbb{E}_{\phi_0 \sim P_0} \Big[\mathcal{L}_{\mathrm{LM}}(\phi_K(\psi); \mathcal{D}_{\mathrm{val}}) \Big].$$

In our project, however, the distilled dataset is not defined by a single fixed optimization rule. Instead, the formal search space has two axes. The first axis is the objective used to optimize ψ:

$$\mathcal{O} \in \{\mathcal{O}_{\mathrm{GM}}, \mathcal{O}_{\mathrm{1step}}, \mathcal{O}_{\mathrm{FM}}, \mathcal{O}_{\mathrm{TM}}\},$$

corresponding to gradient matching, one-step optimization with a learnable learning rate, feature matching, and trajectory matching. The second axis is the parameterization family used to represent the synthetic text:

$$\psi \in \{\Psi_{\mathrm{full}}, \Psi_{\text{top-}k}, \Psi_{\mathrm{anchors}}, \Psi_{\mathrm{concepts}}\}.$$

For each pair of choices, we solve a separate optimization problem

$$\psi^*_{\mathcal{O},\Psi} = \arg\min_{\psi \in \Psi} \mathcal{O}(\psi; \mathcal{D}),$$

and then evaluate the resulting distilled dataset by training a final learner on $\widetilde{\mathcal{D}}_{\psi^*_{\mathcal{O},\Psi}}$ and measuring perplexity on held-out data.

4.2 Causal Language Modeling Task

We formalize the underlying model-training problem as causal language modeling over a vocabulary $\mathcal{V}$. After tokenization, each training example is a sequence of token ids

$$x_i = (w_{i,1}, \ldots, w_{i,T}), \qquad w_{i,t} \in \{0, \ldots, |\mathcal{V}| - 1\}.$$

The dataset returns the same token sequence as both input ids and labels, while the causal language-modeling loss performs the next-token shift internally. A causal Transformer language model f_ϕ maps a sequence prefix to logits

$$z_\phi(x_i) \in \mathbb{R}^{T \times |\mathcal{V}|}, \qquad p_\phi(v \mid w_{i,\le t}) = \mathrm{softmax}(z_{\phi,t}(x_i))_v.$$

Therefore, for hard-token training, positions $1, \ldots, T-1$ are used to predict positions $2, \ldots, T$. If $m_{i,t} \in \{0, 1\}$ is the attention mask, the hard-target loss is

$$\mathcal{L}_{\mathrm{hard}}(\phi; B) = -\frac{\sum_{x_i \in B} \sum_{t=1}^{T-1} m_{i,t+1} \log p_\phi(w_{i,t+1} \mid w_{i,\le t})}{\sum_{x_i \in B} \sum_{t=1}^{T-1} m_{i,t+1}}$$

The same next-token objective is used for soft synthetic data. If a synthetic sequence is represented by token distributions $q_{m,t} \in \Delta^{|\mathcal{V}|}$, the model receives the corresponding soft input embeddings and is trained with the shifted soft-target cross-entropy

$$\mathcal{L}_{\text{soft}}(\phi; \widetilde{B}_\psi) = -\frac{\sum_{\widetilde{x}_m \in \widetilde{B}_\psi} \sum_{t=1}^{T-1} \widetilde{m}_{m,t+1} \sum_{v \in \mathcal{V}} q_{m,t+1,v} \log p_\phi(v \mid \widetilde{x}_{m,\le t})}{\sum_{\widetilde{x}_m \in \widetilde{B}_\psi} \sum_{t=1}^{T-1} \widetilde{m}_{m,t+1}}$$

In our fixed-length experiments the attention mask is identically one, so these expressions reduce to averaging over $|B|(T-1)$ or $|\widetilde{B}_\psi|(T-1)$ prediction positions, matching the implementation. Thus, both real-data training and downstream training on distilled data are instances of the same causal language-modeling task; only the source and parameterization of the training sequences change.

4.3 Distillation Objectives

The four objective families used in the comparison can be written in a common notation. For gradient matching, we compare the training signals induced by real and synthetic batches:

$$\mathcal{O}_{\text{GM}}(\psi) = \sum_{\ell=1}^{L} d\Big(\nabla_{\phi_\ell} \mathcal{L}_{\text{LM}}(\phi; B), \nabla_{\phi_\ell} \mathcal{L}_{\text{LM}}(\phi; \widetilde{B}_\psi)\Big).$$

For the one-step objective, we directly optimize the learner's quality after a single update on the distilled data,

$$\phi_1 = \phi_0 - \eta \nabla_\phi \mathcal{L}_{\text{LM}}(\phi_0; \widetilde{\mathcal{D}}_\psi), \qquad \mathcal{O}_{\text{1step}}(\psi, \eta) = \mathcal{L}_{\text{LM}}(\phi_1; \mathcal{D}_{\text{val}}),$$

where η is treated as a learnable scalar or vector parameter. For feature matching, we compare hidden representations instead of gradients:

$$\mathcal{O}_{\text{FM}}(\psi) = \sum_{\ell=1}^{L} \Big\| \mu_\ell(B) - \mu_\ell(\widetilde{B}_\psi) \Big\|_2^2, \qquad \mu_\ell(B) = \frac{1}{|B|} \sum_{x \in B} h_\ell(\phi; x).$$

For trajectory matching, we optimize the distilled data so that several learner updates on $\widetilde{\mathcal{D}}_\psi$ follow a precomputed expert trajectory:

$$\mathcal{O}_{\text{TM}}(\psi) = \frac{\|\widehat{\phi}_{t+N}(\psi) - \phi^*_{t+M}\|_2^2}{\|\phi^*_t - \phi^*_{t+M}\|_2^2}.$$

4.4 Synthetic Text Parametrization

The parameterization axis determines how each synthetic sequence is represented before it is fed to the learner. Assume that each synthetic sequence has fixed length T. In the full soft-token family, every position carries a full vocabulary distribution:

$$a_{m,t} \in \mathbb{R}^{|\mathcal{V}|}, \qquad p_{m,t} = \text{softmax}(a_{m,t}/\tau), \qquad \widetilde{e}_{m,t} = p_{m,t}\mathbf{E}.$$

In the top-k family, only a small support $\Omega_{m,t} \subset \mathcal{V}$ with $|\Omega_{m,t}| = k$ is active:

$$b_{m,t} \in \mathbb{R}^k, \qquad p_{m,t,\Omega_{m,t}} = \text{softmax}(b_{m,t}/\tau), \qquad p_{m,t,v} = 0 \text{ for } v \notin \Omega_{m,t}.$$

In the anchor family, each position is a mixture over a shared set of anchor distributions:

$$r_j = \mathrm{softmax}(s_j), \quad s_j \in \mathbb{R}^{|\mathcal{V}|}, \qquad \alpha_{m,t} = \mathrm{softmax}(u_{m,t}), \quad u_{m,t} \in \mathbb{R}^K,$$

$$p_{m,t} = \sum_{j=1}^{K} \alpha_{m,t,j} r_j, \qquad \widetilde{e}_{m,t} = p_{m,t}\mathbf{E}.$$

In the concept family, the position is represented in a lower-dimensional learned basis:

$$z_{m,t} \in \mathbb{R}^K, \qquad C \in \mathbb{R}^{K \times d}, \qquad \widetilde{e}_{m,t} = z_{m,t}C.$$

To obtain hard tokens from a continuous representation, we either decode by nearest-neighbor search in the embedding space or by an explicit decoder back into the vocabulary.

Table 1: Comparison of synthetic text parameterizations. Here, M is the number of synthetic texts, T is sequence length, $|\mathcal{V}|$ is vocabulary size, d is embedding dimension, K is the number of anchors or concepts, and k is the sparse support size.

Method	Learned representation	Number of parameters	Decoding
Full soft-token	full logit vector at each position	$O(MT\|\mathcal{V}\|)$	$\arg\max p_{m,t}$
Top-k	k logits per position + support $\Omega_{m,t}$	$O(MTk)$	$\arg\max$ within $\Omega_{m,t}$
Anchors	K shared soft-token anchors + mixtures	$O(K\|\mathcal{V}\| + MTK)$	$\arg\max p_{m,t}$
Concepts	coordinates in a concept basis	$O(Kd + MTK)$ or $O(K\|\mathcal{V}\| + MTK)$ with decoder	nearest token or decoder

This formalization makes the project objective explicit. We are not searching only for a single best distilled dataset, but for a clear empirical comparison across the Cartesian product of objective families and synthetic-text parameterizations.

5 Experimental Setup

5.1 Dataset

The experiments are based on TinyStories. For reproducibility, all local-text experiments use a fixed source file built from the first 15 000 texts of the TinyStories Hugging Face training split. We partition this file in source order into 10 000 train texts, 3 000 validation texts, and 2 000 test texts.

Texts are tokenized with a locally trained BPE tokenizer. The tokenizer uses byte-level pre-tokenization, NFC normalization, and three special tokens: [PAD], [EOS], and [UNK]. In the tokenizer used for the main reported experiments, the vocabulary size is 1024, and the special token ids are zero-based: [PAD] = 0, [EOS] = 1, and [UNK] = 2.

For language-model training, the tokenizer appends an end-of-sequence token to each text. The resulting token stream is split into fixed-length chunks. The dataset returns each chunk as both `input_ids` and `labels`, while the causal language-modeling loss performs the next-token shift internally. Since the chunks used in the reported experiments are fixed-length and contain no padding, the returned `attention_mask` is all ones. We use `seq_len` = 64 for short-sequence experiments and `seq_len` = 256 for longer full-story experiments.

5.2 Evaluation Metrics

Because the initial training setup is based on TinyStories, the main evaluation metric should be perplexity of the final model trained on the distilled dataset and evaluated on the held-out split described above. In other words, we primarily care about how well the distilled data support next-token prediction on this fixed held-out portion of the original distribution. For a model f_ϕ, this can be measured through

$$\mathrm{PPL} = \exp\left(-\frac{1}{\sum_{i=1}^{N}(T_i-1)}\sum_{i=1}^{N}\sum_{t=1}^{T_i-1}\log p_\phi(w_{i,t+1} \mid w_{i,\le t})\right),$$

where lower perplexity indicates better language-modeling quality. This is the main metric because it directly matches the training objective on TinyStories and gives a natural measure of how much of the original dataset structure is preserved by the distilled set.

In addition to raw perplexity, we also report the distillation ratio

$$\rho = \frac{Q(\text{train on } \widetilde{\mathcal{D}}) - Q(\text{random guess})}{Q(\text{train on } \mathcal{D}) - Q(\text{random guess})}.$$

Here Q is a generic quality measure, and in our setting it can be instantiated as negative perplexity or another monotone transformation of perplexity so that larger values correspond to better performance. This metric is useful because it normalizes the result relative to the full-data upper bound and gives a more interpretable notion of how much of the original training signal is retained by the distilled dataset.

We also want to track efficiency, but in this proposal it is more natural to describe it as a group of supporting measurements rather than as separate checklist items. In particular, we will record the number of trainable parameters in the synthetic dataset, peak GPU memory during distillation, total wall-clock training time, the runtime of one outer optimization step, and the size of the saved distilled dataset on disk. These measurements will help us judge whether a given parameterization offers a practically meaningful improvement rather than only a small gain in final perplexity.

5.3 Conducted Experiments

We already implemented almost all main parts of the project, and in some directions we did a bit more than in the initial plan. We implemented the full basic pipeline: training on real TinyStories data, optimizing a distilled synthetic dataset, training a downstream model on distilled data, and precomputing expert trajectories for trajectory matching.

The implemented code already covers both axes from the proposal. On the objective side, we implemented one-step optimization, gradient matching (GM), feature matching (FM), and trajectory matching (TM). On the representation side, we implemented full soft-token optimization, top-k parameterizations, some anchor variants, and concept-based parameterizations. However, we do not claim that we already finished the full detailed comparative analysis for all these variants. At the current stage, the main goal was to build and debug the codebase, run the first working experiments, and obtain the first intermediate results.

The experimental workflow is as follows:

1. prepare TinyStories text and train a local BPE tokenizer;

2. train a real-data baseline language model;

3. run distillation for a chosen objective and synthetic-text parameterization;

4. train a downstream model on the saved distilled dataset and evaluate perplexity on held-out TinyStories data.

At the same time, we are still in the stage of debugging and stabilizing the full experimental code. So the implementation is already broad, but not all runs are fully checked and cleaned yet. Because of this, the current results should be treated as intermediate results from the implemented system.

The main reported setup uses the dataset and tokenizer described above. The main metric is downstream validation perplexity of a small causal language model trained on real or distilled data. We compare short-sequence experiments with `seq_len` $= 64$ and longer full-story experiments with `seq_len` $= 256$.

So, at this milestone we already have meaningful partial results. They are enough to compare objectives, compact parameterizations, and different distilled dataset sizes, and to see which ideas look more promising.

6 Preliminary Results

6.1 Updated Experimental Results

After additional debugging of the training and evaluation pipeline, we found and corrected a number of small implementation issues. These fixes substantially changed the quantitative picture, most noticeably improving the real-data baseline. Therefore, the results in this section should be treated as the current valid comparison and should supersede the preliminary numbers reported earlier. All main runs in Table 2 use the same TinyStories setup: a local BPE tokenizer with vocabulary size 1024, fixed train/validation/test rows $0, \dots, 9999$, $10000, \dots, 12999$, and $13000, \dots, 14999$, respectively, and a causal Transformer with `max_seq_len` $= 256$, $d_{\text{model}} = 128$, 4 attention heads, 4 layers, and no dropout. Unless stated otherwise, the distilled dataset contains $N = 256$ synthetic sequences of length 256. For soft checkpoints, downstream training is performed on soft-token synthetic data, while validation and test perplexities are always measured on held-out real hard-token TinyStories rows.

The main conclusion from the updated runs is more conservative than in the previous version. The distilled datasets are useful, but the gap to the full real-data baseline remains large. The best current synthetic results reach test perplexity around 61.5, while training on $10,000$ real TinyStories rows reaches test perplexity 21.777. Thus, the present experiments show a nontrivial soft-token distillation signal, but they do not yet match the quality of training on the full real dataset.

The strongest synthetic methods are very close to each other. Trajectory matching, gradient matching, and both one-step variants all reach nearly the same best test perplexity, between 61.467 and 61.513. This suggests that, after fixing the pipeline, the choice among these objectives is less decisive in the current configuration than it appeared in the earlier intermediate results. Feature matching remains noticeably weaker: its best test perplexity is 74.851, which is substantially worse than the best GM, TM, and one-step runs under the same basic setup.

The fixed top-k result is also important. With only $k = 8$ candidates per position, one-step optimization reaches best test perplexity 61.922, which is very close to the full soft-token runs. This shows that a much more compact support can preserve most of the performance of the full-vocabulary soft-token representation. At the same time, this result should not be overinterpreted as a solution to discrete text distillation: in the current run, the entropy of the top-k distributions stays almost zero, so the learned object behaves more like a hard-token-like synthetic set inside a fixed support than like a smoothly optimized distribution over a broad vocabulary.

Another practical observation is overfitting during downstream evaluation. For the strongest synthetic runs, the best validation epoch is around epoch 6, while the final epoch has substantially worse test perplexity. For example, the TM run reaches best test perplexity 61.467, but its final test perplexity is 104.729. Therefore, comparisons in Table 2 should be based on the best validation epoch rather than on the last epoch, and future sweeps should use early stopping or shorter evaluation schedules.

Setup	Objective	Representation	Best Val PPL	Best Test PPL	Final Test PPL	Interpretation
Real data, $10,000$ rows	—	real text	23.310	21.777	21.777	full-data upper reference, not distilled
Distilled data, $N = 256$	TM	full soft-token	64.138	61.467	104.729	best synthetic run so far
Distilled data, $N = 256$	one-step, fixed LR	full soft-token	64.149	61.470	105.042	almost identical to TM at the best epoch
Distilled data, $N = 256$	GM	full soft-token	64.149	61.470	104.575	very close to TM and one-step optimization
Distilled data, $N = 256$	one-step, learned LR	full soft-token	64.191	61.513	104.611	learned inner learning rate gives similar quality
Distilled data, $N = 256$	one-step, learned LR	fixed top-k, $k = 8$	63.980	61.922	106.590	compact representation with almost full-soft quality
Distilled data, $N = 256$	FM	full soft-token	77.976	74.851	136.303	weaker than GM, TM, and one-step objectives in this setup

Table 2: Updated main comparable results. Lower perplexity is better. "Best" values are taken at the best validation epoch, while the final test value is reported at the last evaluation epoch.

6.2 Soft-to-Hard Gap

The results above are obtained with soft-token synthetic data. This is useful for testing whether the distilled representations contain a training signal, but it is not the final form we would like to obtain. The broader goal is to produce ordinary text sequences that can be stored, inspected, and reused without requiring soft distributions over the vocabulary. A natural first conversion rule is hard decoding by the most likely token at each position,

$$\hat{x}_{m,t} = \arg\max_{v \in \mathcal{V}} p_{m,t,v}.$$

However, this simple discretization currently creates a large soft-to-hard gap. When we train on the hard sequences obtained as arg max of the learned soft tokens, the resulting quality is still much worse than the corresponding real-data subset baseline. This is an important limitation of the current approach: the optimized soft tokens are effective as continuous training objects, but their direct hard-token projection does not yet provide competitive discrete synthetic text.

For this reason, the next stage of the project is not only to improve soft-token perplexity, but also to reduce the degradation caused by discretization. We have started experiments with schedules for k in the top-k parameterization, with the goal of gradually controlling the amount of discrete support available during optimization. So far, these experiments have not produced a clear improvement. At the current stage, the safest interpretation is that top-k sparsity can make the representation compact without strongly hurting soft-token performance, but it does not by itself solve the problem of obtaining high-quality ordinary text from the learned soft tokens.

Overall, the updated experiments establish a clearer baseline for the rest of the work. The implementation fixes make the real-data reference substantially stronger, the best soft-token distilled runs are now clustered around similar test perplexity, and the main unresolved issue is the conversion from optimized soft representations to useful discrete text.

7 Future Plans

We also outline several next directions that we would like to explore after the current stage of implementation and debugging.

7.1 Visualization and Interpretation

- **Evolution Heatmaps:** For parameterizations based on full soft-token distributions or top-k distributions, we plan to visualize how the probability mass over the vocabulary changes during training. Such heatmaps can show whether the optimization starts from a diffuse distribution and gradually moves toward sharper token choices. This may also help us compare how quickly different parameterizations become interpretable or decodable.

- **Concept Space Projections:** For concept-based parameterizations, we plan to visualize the learned concept vectors C together with the frozen token embeddings. We can use dimensionality reduction methods such as UMAP or t-SNE for this purpose. These visualizations may help us understand whether the learned concepts correspond to meaningful semantic or syntactic regions in the embedding space. For example, we may observe whether some concepts are close to groups of nouns, verbs, function words, or semantically related tokens.

- **Decoding and Manual Inspection:** We also plan to inspect the nearest discrete tokens corresponding to the learned soft representations. Although soft tokens are not always directly interpretable as normal text, nearest-neighbor decoding can provide useful qualitative evidence. This analysis can help us see whether the distilled representations form coherent pseudo-sentences, repeated patterns, or abstract combinations of words that are useful only for optimization.

7.2 Hybrid Parameterizations

The parameterizations considered so far are studied mostly as separate alternatives. However, we believe that combining their mechanisms may lead to better performance. In particular, one promising direction is to develop a hybrid method that combines the *Top-k* and *Concept-based* approaches.

The motivation for this idea is that these two methods have complementary advantages. The top-k parameterization encourages sparsity and makes the learned representation closer to a small set of real vocabulary items. This can improve interpretability and make the distilled tokens easier to decode. On the other hand, concept-based parameterizations can represent more abstract information by learning vectors that are not restricted to individual tokens. This may help capture broader semantic patterns that are shared across multiple words.

7.3 Towards Robust and Model-Agnostic Distilled Datasets

A common limitation of dataset distillation is that the synthetic data may overfit to the specific model used during optimization. In other words, the distilled dataset may exploit the particular architecture, initialization, or training dynamics of one model rather than learning information that is useful more generally. This issue is especially important for language models, because different architectures may use the same data in different ways.

To address this limitation, we plan to investigate approaches that make the distilled datasets more model-agnostic. Our goal is to produce synthetic data that can be reused across different models without requiring a completely new distillation process. We will explore two main strategies:

1. **Cross-Model Ensemble Optimization:** Instead of optimizing the synthetic dataset using only one proxy model, we plan to use an ensemble of several language models. During each distillation step, the synthetic data would be evaluated on multiple architectures, and the final objective would aggregate their losses or gradients. This should encourage the distilled dataset to capture more general linguistic features, rather than model-specific shortcuts. We expect that this approach may improve transferability, although it will also increase the computational cost.

2. **Meta-Learned Base Datasets:** We also plan to explore a meta-learning formulation inspired by Model-Agnostic Meta-Learning (MAML) [2]. In this setting, the goal is not necessarily to distill a dataset that directly trains any model from scratch. Instead, we would optimize a base distilled dataset that can be quickly adapted to a new language model using only a small number of fine-tuning steps. This two-stage formulation may make the distilled data more robust and reusable across different architectures and training settings.

Overall, these future directions define the main possible extensions of our work. Since the available time and computational resources are limited, we do not guarantee that all of them will be completed. We will proceed step by step, prioritizing the most important experiments first, and then moving to more advanced extensions if time permits.

References

[1] George Cazenavette, Tongzhou Wang, Antonio Torralba, Alexei A. Efros, and Jun-Yan Zhu. Dataset distillation by matching training trajectories. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10718–10727, 2022. doi: 10.1109/CVPR52688.2022.01045. URL https://openaccess.thecvf.com/content/CVPR2022/html/Cazenavette_Dataset_Distillation_by_Matching_Training_Trajectories_CVPR_2022_paper.html.

[2] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pp. 1126–1135. PMLR, 2017.

[3] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *International Conference on Learning Representations*, 2017. URL https://openreview.net/forum?id=rkE3y85ee.

[4] Wei Jin, Lingxiao Zhao, Shichang Zhang, Yozen Liu, Jiliang Tang, and Neil Shah. Graph condensation for graph neural networks. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=WLEx3Jo4QaB.

[5] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 3045–3059. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.243. URL https://aclanthology.org/2021.emnlp-main.243/.

[6] Jonathan Light, Shie Mannor, Aviv Tamar, and Gal Chechik. Dataset distillation for offline reinforcement learning. *arXiv preprint arXiv:2407.20299*, 2024. URL https://arxiv.org/abs/2407.20299.

[7] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. In *International Conference on Learning Representations*, 2017. URL https://openreview.net/forum?id=S1jE5L5gl.

[8] Aru Maekawa, Satoshi Kosugi, Kotaro Funakoshi, and Manabu Okumura. DiLM: Distilling dataset into language model for text-level dataset distillation. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 3138–3153. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-naacl.199. URL https://aclanthology.org/2024.findings-naacl.199/.

[9] Ben Peters, Vlad Niculae, and André F. T. Martins. Sparse sequence-to-sequence models. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 1504–1519. Association for Computational Linguistics, 2019. doi: 10.18653/v1/P19-1146. URL https://aclanthology.org/P19-1146/.

[10] Cheng Shen, Yew-Soon Ong, and Joey Tianyi Zhou. CondenseLM: LLMs-driven text dataset condensation via reward matching. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 1237–1252. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.65. URL https://aclanthology.org/2025.emnlp-main.65/.

[11] Ilia Sucholutsky and Matthias Schonlau. Soft-label dataset distillation and text dataset distillation. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8. IEEE, 2021. doi: 10.1109/IJCNN52387.2021.9533769. URL https://arxiv.org/abs/1910.02551.

[12] Yefan Tao, Luyang Kong, Andrey Kan, and Laurent Callot. Textual dataset distillation via language model embedding. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 12557–12569. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-emnlp.733. URL https://aclanthology.org/2024.findings-emnlp.733/.

[13] Tongzhou Wang, Jun-Yan Zhu, Antonio Torralba, and Alexei A. Efros. Dataset distillation. *arXiv preprint arXiv:1811.10959*, 2018. URL https://arxiv.org/abs/1811.10959.

[14] Bo Zhao and Hakan Bilen. Dataset condensation with distribution matching. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 6503–6512, 2023. URL https://openaccess.thecvf.com/content/WACV2023/html/Zhao_Dataset_Condensation_With_Distribution_Matching_WACV_2023_paper.html.

[15] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. Dataset condensation with gradient matching. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=mSAKhLYLSsl.